

Spring Data JPA & Hibernate Avançado

O Guia Prático, Conceitual e Arquitetural Definitivo: Ciclo de Vida de Entidades, Dicionário de Anotações, Relacionamentos Complexos, Performance contra N+1, Derived Queries, Transações e o Blueprint Reutilizável em Qualquer Projeto.

A Trindade: JDBC, JPA & Hibernate

A separação exata de papéis entre a especificação Jakarta, a engine de ORM e o framework Spring Data.

Dicionário Exhaustivo de Anotações

Mapeamento de chaves primárias, estratégias de geração, colunas, enums, embeddables, herança e auditoria.

Relacionamentos & Performance

Como modelar 1:N, N:1, 1:1 e N:N evitando o temido problema N+1 com FetchType.LAZY e JOIN FETCH.

Transações & Dirty Checking

O poder do @Transactional, otimização de leitura com readOnly e sincronização automática de estado com o banco.

Sumário Executivo do Manual

Capítulo 1: Fundamentos da Persistência Java (JDBC vs JPA vs Hibernate vs Spring Data)	03
Capítulo 2: O Ciclo de Vida de uma Entidade JPA & O Efeito Dirty Checking	05
Capítulo 3: Dicionário Completo de Anotações JPA & Hibernate (Código Comentado)	07
3.1. Estrutura Básica e Tabela: <code>@Entity</code> , <code>@Table</code>	07
3.2. Chaves Primárias: <code>@Id</code> , <code>@GeneratedValue</code> (IDENTITY, SEQUENCE, UUID)	08
3.3. Colunas e Tipos: <code>@Column</code> , <code>@Enumerated</code> , <code>@Lob</code> , <code>@Transient</code>	10
3.4. Objetos Embutidos: <code>@Embeddable</code> , <code>@Embedded</code> , <code>@AttributeOverride</code>	12
3.5. Auditoria & Concorrência: <code>@CreationTimestamp</code> , <code>@UpdateTimestamp</code> , <code>@Version</code>	13
3.6. Herança em JPA: <code>@MappedSuperclass</code> vs Estratégias de Herança	14
Capítulo 4: Relacionamentos entre Entidades (A Arte do ORM)	16
4.1. Os 4 Tipos: <code>@ManyToOne</code> , <code>@OneToMany</code> , <code>@OneToOne</code> , <code>@ManyToMany</code>	16
4.2. O Problema N+1 e a Guerra: <code>FetchType.LAZY</code> vs <code>EAGER</code>	19
4.3. Estratégias de Cascata (<code>CascadeType</code>) & <code>orphanRemoval</code>	21
4.4. A Armadilha do Lombok <code>@Data</code> em Relacionamentos	22
Capítulo 5: Spring Data Repositories: Do Básico ao Modo Ninja	23
5.1. A Hierarquia: <code>Repository</code> até <code>JpaRepository</code>	23
5.2. Derived Query Methods (Palavras-Chave de Ouro)	24
5.3. Paginação e Ordenação: <code>Pageable</code> , <code>Page</code> vs <code>Slice</code>	26
5.4. Consultas Customizadas com <code>@Query</code> (JPQL vs SQL Nativo)	28
5.5. Operações de Modificação em Massa: <code>@Modifying</code>	30
5.6. Projeções de Alta Performance (DTOs e Interfaces)	31
Capítulo 6: Gestão Transacional com <code>@Transactional</code>	32
Capítulo 7: Anatomia Prática do SIGEP (Estudo de Caso Real)	34
Capítulo 8: Blueprint Universal: Como Modelar Persistência em Qualquer Projeto	36
Capítulo 9: As 4 Perguntas de Ouro do Arquiteto	38

Capítulo 1: Fundamentos da Persistência Java

Para dominar o Spring Data JPA, é essencial entender o papel de cada camada tecnológica envolvida. Muitos desenvolvedores utilizam JPA, Hibernate e Spring Data como se fossem a mesma coisa, mas são quatro camadas completamente distintas que trabalham em conjunto:

Camada	O que é?	Papel na Arquitetura
1. JDBC (Java Database Connectivity)	Driver / API de baixo nível nativa do Java.	Abre a conexão TCP/IP com o PostgreSQL/MySQL, envia strings SQL puras e recebe tabelas brutas (<code>ResultSet</code>). Código verboso, propenso a erros de SQL Injection e conexão vazada.
2. JPA (Jakarta Persistence API)	Especificação oficial (Interfaces e Regras).	Define como o Java deve mapear classes para tabelas (ORM). A JPA não possui código executável ; ela é apenas um contrato contendo anotações (<code>@Entity</code> , <code>@Id</code> , etc.) e interfaces (<code>EntityManager</code>).
3. Hibernate	Implementação concreta da especificação JPA.	O "motor" real por baixo dos panos. É o Hibernate quem lê as anotações JPA, gera o SQL dinâmico específico para o seu banco (dialetos PostgreSQL), gerencia o cache e mapeia objetos.
4. Spring Data JPA	Camada de Abstração do Spring.	Elimina a necessidade de você interagir diretamente com o <code>EntityManager</code> do Hibernate. Cria automaticamente implementações de CRUD, paginação e queries a partir de interfaces declarativas (<code>JpaRepository</code>).

FLUXO DE EXECUÇÃO DA PERSISTÊNCIA:

Seu Código (Service): Chama `usuarioRepository.save(usuario)`



Spring Data JPA: Intercepta a interface via Proxy e chama o `EntityManager`



Hibernate (Engine): Faz o Dirty Checking, valida a entidade e traduz para `INSERT INTO usuarios...`



Driver JDBC: Envia os bytes do comando SQL para o banco PostgreSQL



PostgreSQL: Executa a transação no disco e confirma com COMMIT

Capítulo 2: O Ciclo de Vida de uma Entidade & Dirty Checking

O Hibernate não trata todos os objetos da mesma forma. No momento em que você dá um `new Usuario()`, ele é apenas um pedaço de memória Java. Para entender como o JPA salva e atualiza dados, você deve conhecer os **4 estados do ciclo de vida de uma entidade**:

Estado	Definição	Comportamento com o Banco
Transient (Novo)	Instanciado via <code>new</code> na memória. Não possui ID de banco e não está ligado ao EntityManager.	O banco de dados não sabe que esse objeto existe. Se a aplicação morrer, os dados somem.
Managed (Gerenciado / Persistente)	Possui ID no banco e está associado a uma sessão ativa do EntityManager (geralmente dentro de um método <code>@Transactional</code>).	Qualquer alteração feita nos atributos do objeto é monitorada automaticamente! Ao final da transação, o Hibernate gera o <code>UPDATE</code> no banco sozinho.
Detached (Destacado)	Possui ID e já existe no banco, mas a transação que o carregou foi encerrada (o EntityManager fechou).	Se você alterar um atributo dele agora, o banco NÃO será atualizado, a menos que você chame <code>repository.save(objeto)</code> (que internamente chama <code>entityManager.merge()</code>).
Removed (Removido)	Objeto que estava gerenciado e foi marcado para deleção através de <code>repository.delete()</code> .	Ao finalizar a transação (commit), o comando <code>DELETE FROM ...</code> é enviado ao banco.

2.1. O Poder do Dirty Checking (Verificação Automática de Estado Sujo)

Este é um dos recursos mais elegantes e que mais causa confusão em desenvolvedores iniciantes. Veja este exemplo clássico de um método em uma classe Service:

```
@Service
@RequiredArgsConstructor
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    @Transactional // ← ATENÇÃO AQUI! Abre transação e mantém o EntityManager ativo
    public void atualizarNome(Long id, String novoNome) {
        // 1. O findById carrega o usuário e coloca ele no estado MANAGED
        Usuario usuario = usuarioRepository.findById(id)
            .orElseThrow(() -> new EntityNotFoundException("Usuário não encontrado"));

        // 2. Alteramos o estado em memória
        usuario.setNome(novoNome);

        // PERGUNTA: PRECISA DE usuarioRepository.save(usuario) AQUI?
        // RESPOSTA: NÃO! O Dirty Checking do Hibernate faz o UPDATE sozinho!
    }
}
```

Como o Dirty Checking Funciona por Baixo dos Panos: Quando o Hibernate busca o objeto no banco dentro de um método `@Transactional`, ele guarda uma *fotografia original* (snapshot) dos dados do objeto. Quando o método termina e a transação está prestes a fazer o commit, o Hibernate compara o objeto atual com o snapshot original. Se algum campo foi alterado (está "sujo" / dirty), ele emite o comando `UPDATE usuarios SET nome = ? WHERE id = ?` automaticamente!

Capítulo 3: Dicionário Completo de Anotações JPA & Hibernate

Aqui está a enciclopédia definitiva de todas as anotações que você usará no dia a dia como engenheiro de software Java, com código ilustrativo, linha por linha explicada e as razões arquiteturais de cada escolha.

3.1. Anotações Estruturais e de Tabela

```
@Entity // 1. Identifica a classe como entidade gerenciada pelo JPA (tabela no banco)
@Table(
    name = "tb_processos", // 2. Nome real da tabela no banco de dados
    uniqueConstraints = {
        // 3. Restrição de unicidade composta (ex: não pode haver mesma sigla no mesmo órgão)
        @UniqueConstraint(name = "uk_setor_orgao_sigla", columnNames = {"orgao_id", "sigla"})
    },
    indexes = {
        // 4. Criação de índice explícito para acelerar consultas pesadas no PostgreSQL
        @Index(name = "idx_processo_protocolo", columnList = "numero_protocolo")
    }
)
public class Processo { ... }
```

AS 4 PERGUNTAS: @ENTITY & @TABLE

Para que serve? Mapeia a classe Java para uma tabela física no banco relacional e configura restrições (chaves únicas e índices).

Para que usar? Para padronizar nomes de tabelas (ex: usar snake_case e plural ou prefixo `tb_`) sem alterar o nome da classe Java.

Quando usar? Em toda classe de domínio que precise ser persistida no banco relacional.

Devo usar em outros projetos? Sim, é a base mandatória de qualquer entidade JPA.

3.2. Chaves Primárias: @Id e Estratégias de Geração (@GeneratedValue)

Toda entidade JPA exige uma Chave Primária (Primary Key). Existem quatro estratégias de geração disponíveis:

Estratégia	Como funciona?	Quando usar / Vantagens
<code>GenerationType.IDENTITY</code>	Delega a geração para a coluna auto-incremento nativa do banco (<code>SERIAL</code> ou <code>BIGSERIAL</code> no PostgreSQL, <code>AUTO_INCREMENT</code> no MySQL).	A mais simples e comum para a maioria dos sistemas. <i>Desvantagem:</i> Impede que o Hibernate faça inserções em lote (JDBC Batch) automáticas sem antes fazer o INSERT para descobrir o ID.
<code>GenerationType.SEQUENCE</code>	Utiliza um objeto de sequência próprio do banco (<code>CREATE SEQUENCE seq_usuario</code>) que aloca blocos de IDs na memória antes de gravar.	Recomendado para altíssimo desempenho e processamento em lote (Batching) no PostgreSQL e Oracle. O Hibernate busca 50 IDs de uma vez na sequência e não trava esperando o banco.
<code>GenerationType.AUTO</code>	O Hibernate escolhe a estratégia que achar melhor baseado no dialeto do banco configurado.	Não recomendado em produção! Pode escolher SEQUENCE em um banco e TABLE em outro, gerando comportamentos inesperados.
UUID (Identificador Único Universal)	Gera strings aleatórias globais de 128 bits (ex: <code>c8a1b2c3-4d5e- ...</code>).	Perfeito para arquiteturas distribuídas e microsserviços , pois IDs podem ser gerados no cliente ou em múltiplos servidores sem risco de colisão de ID.

```
// EXEMPLO 1: Estratégia IDENTITY (Padrão do seu projeto SIGEP)
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// EXEMPLO 2: Estratégia SEQUENCE de Alta Performance (PostgreSQL)
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "seq_pedidos")
@SequenceGenerator(name = "seq_pedidos", sequenceName = "pedidos_id_seq", allocationSize = 50)
private Long id;

// EXEMPLO 3: UUID Nativo no Spring Boot 3+ / Java 21
@Id
@GeneratedValue(strategy = GenerationType.UUID)
private UUID id;
```

3.3. Mapeamento de Colunas e Tipos Básicos

```
public class ExemploColunas {

    // 1. Configurações avançadas de coluna
    @Column(
        name = "cpf_cnpj",           // Nome da coluna no banco
        length = 14,                 // VARCHAR(14)
        nullable = false,            // NOT NULL
        unique = true,               // Garante constraint de unicidade
        updatable = false,           // Impede que o Hibernate altere este campo em UPDATES!
        insertable = true            // Inclui na instrução INSERT
    )
    private String cpfCnpj;

    // 2. Valores Decimais Monetários / Precisão Exata
    @Column(name = "valor_total", precision = 12, scale = 2, nullable = false)
    private BigDecimal valorTotal; // DECIMAL(12, 2) no banco

    // 3. Enums no Banco: SEMPRE use EnumType.STRING!
    @Enumerated(EnumType.STRING)
    @Column(name = "status", length = 20, nullable = false)
    private StatusProcesso status;

    // 4. Textos Longos (CLOB) ou Arquivos Binários/PDFs (BLOB)
    @Lob
    @Column(name = "termo_referencia", columnDefinition = "TEXT")
    private String textoGigante;

    @Lob
    @Column(name = "anexo_pdf")
    private byte[] arquivoPdf;

    // 5. Campos em Memória que NÃO viram coluna no banco de dados
    @Transient
    private String tokenTemporarioEmMemoria;
}
```

Perigo Fatal com @Enumerated(EnumType.ORDINAL): Por padrão, se você não colocar nada, o JPA usa `ORDINAL`. Isso salva o índice numérico do Enum (0, 1, 2...). Se um desenvolvedor adicionar um novo valor no início ou no meio do Enum meses depois, todos os dados existentes no seu banco de dados ficarão trocados e corrompidos! Use SEMPRE `EnumType.STRING`.

3.4. Objetos Embutidos: @Embeddable e @Embedded (Design DDD)

Às vezes você tem dados que pertencem conceitualmente à mesma tabela do banco, mas na orientação a objetos você quer organizá-los em uma classe separada (conhecido como *Value Object* no Domain-Driven Design).

```

// 1. A classe que define os campos embutíveis
@Embeddable
@Data
public class Endereco {
    @Column(name = "logradouro", length = 100)
    private String logradouro;

    @Column(name = "numero", length = 10)
    private String numero;

    @Column(name = "cep", length = 8)
    private String cep;
}

// 2. A entidade que embute o endereço na sua própria tabela
@Entity
@Table(name = "orgaos")
public class Orgao {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;

    @Embedded // Logradouro, numero e cep viram colunas normais dentro da tabela "orgaos"!
    private Endereco endereco;
}

```

3.5. Auditoria Automática & Concorrência Otimista (@Version)

```
public abstract class EntidadeBaseAuditavel {  
  
    // 1. Auditoria com anotações do Hibernate (Mais simples, sem configuração extra)  
    @CreationTimestamp  
    @Column(name = "data_criacao", nullable = false, updatable = false)  
    private LocalDateTime dataCriacao;  
  
    @UpdateTimestamp  
    @Column(name = "data_atualizacao", nullable = false)  
    private LocalDateTime dataAtualizacao;  
  
    // 2. CONTROLE DE CONCORRÊNCIA OTIMISTA (Optimistic Locking)  
    @Version  
    private Long versao;  
}
```

Para que serve o @Version ?

Imagine dois operadores abrindo o mesmo processo ao mesmo tempo na tela. O Operador A clica em "Salvar" às 10:00:00. O Operador B clica em "Salvar" às 10:00:01 sem saber que A já alterou os dados. Sem o @Version, o Operador B sobrescreve cegamente o trabalho de A (*Lost Update Problem*).

Com o @Version, toda vez que uma linha é alterada, o Hibernate incrementa o número da versão (1 → 2). Quando o Operador B tenta salvar enviando a versão 1 antiga, o Hibernate detecta a colisão e lança `OptimisticLockException`, impedindo a perda de dados!

3.6. Herança em JPA: @MappedSuperclass

Em vez de repetir campos como `id`, `dataCriacao` e `ativo` em todas as 20 entidades do seu projeto, você cria uma classe base abstrata e anota com `@MappedSuperclass`:

```
@MappedSuperclass // Diz ao JPA: "Herde estas colunas para as filhas, mas NÃO crie uma tabela própria para mim"  
@Getter @Setter  
public abstract class EntidadeBase {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @CreationTimestamp  
    @Column(name = "criado_em", updatable = false)  
    private LocalDateTime criadoEm;  
  
    @Column(name = "ativo", nullable = false)  
    private boolean ativo = true;  
}  
  
// As entidades do seu sistema herdam tudo limpo e sem repetição de código:  
@Entity  
@Table(name = "setores")  
public class Setor extends EntidadeBase {  
    private String nome;  
    private String sigla;  
}
```

Capítulo 4: Relacionamentos entre Entidades & Performance

Mapear relacionamentos entre tabelas é o coração do desenvolvimento backend. No entanto, mapeamentos mal feitos são os maiores responsáveis por lentidão e quedas de servidores em produção.

4.1. Os 4 Tipos de Relacionamentos no Modelo Relacional

1. @ManyToOne (Muitos para Um) - O Relacionamento Fundamental

Representa uma Chave Estrangeira (Foreign Key) na tabela atual. É o relacionamento mais eficiente e seguro do JPA.

```
@Entity
@Table(name = "processos")
public class Processo {
    ...
    // MUITOS processos pertencem a UM setor
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "setor_atual_id", nullable = false)
    private Setor setorAtual;
}
```

2. @OneToMany (Um para Muitos) - O Lado Inverso

Representa uma coleção de filhos dentro da classe pai. O atributo `mappedBy` é **obrigatório** para avisar ao JPA quem é o dono da Foreign Key é a classe filha.

```
@Entity
@Table(name = "setores")
public class Setor {
    ...
    // UM setor possui MUITOS processos associados
    @OneToMany(mappedBy = "setorAtual", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    private List<Processo> processos = new ArrayList<>();
}
```

3. @OneToOne (Um para Um)

Uma linha na tabela A aponta para exatamente uma linha na tabela B (ex: Usuário e DetalhesDePerfil).

```
@Entity
public class Usuario {
    @OneToOne(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JoinColumn(name = "configuracao_id", unique = true)
    private ConfiguracaoUsuario configuracao;
}
```

4. @ManyToMany (Muitos para Muitos)

Exige uma tabela intermediária no banco de dados para ligar as duas pontas.

```
@ManyToMany(fetch = FetchType.LAZY)
@JoinTable(
    name = "usuario_grupos", // Tabela associativa intermediária
    joinColumns = @JoinColumn(name = "usuario_id", // FK desta entidade
    inverseJoinColumns = @JoinColumn(name = "grupo_id") // FK da outra entidade
)
private Set<Grupo> grupos = new HashSet<>();
```


4.2. O Problema N+1 e a Guerra: FetchType.LAZY VS FetchType.EAGER

O problema do N+1 é a causa número 1 de APIs Spring Boot ficarem lentas e derrubarem bancos de dados.

O Cenário do Desastre (EAGER):

Imagine que na sua classe `Processo` você colocou `@ManyToOne(fetch = FetchType.EAGER)` no responsável. Agora você executa uma consulta simples para listar 100 processos:

```
List<Processo> processos = processoRepository.findAll();
```

O que o Hibernate faz por baixo dos panos?

1. Executa 1 **query** para buscar os 100 processos: `SELECT * FROM processos;`
2. Para CADA UM dos 100 processos retornados, ele dispara **mais uma query** para buscar o usuário:

```
SELECT * FROM usuarios WHERE id = 1;  
SELECT * FROM usuarios WHERE id = 2;  
... (mais 98 queries!)
```

Para buscar 100 processos, sua aplicação disparou **101 requisições no PostgreSQL!** Em uma tabela de 10.000 registros, sua aplicação dispara 10.001 queries e trava a CPU do banco.

A Solução Definitiva em 2 Passos:

1. **Passo 1:** Declare SEMPRE `FetchType.LAZY` em absolutamente todos os relacionamentos (`@ManyToOne` , `@OneToMany` , `@OneToOne` , `@ManyToMany`). No modo LAZY, o Hibernate só busca o objeto relacionado se você explicitamente invocar `processo.getResponsavel()` .
2. **Passo 2:** Quando você realmente precisar carregar os dois objetos juntos em uma tela, use `JOIN FETCH` na query do Repository:

```
// Dispara EXATAMENTE 1 ÚNICA QUERY COM INNER JOIN NO BANCO!  
@Query("SELECT p FROM Processo p JOIN FETCH p.responsavel JOIN FETCH p.setorAtual")  
List<Processo> buscarProcessosCompleto();
```

Atenção com o Default do JPA:

- `@ManyToOne` e `@OneToOne` por padrão são **EAGER!** Você DEVE mudar explicitamente para `fetch = FetchType.LAZY` .
- `@OneToMany` e `@ManyToMany` por padrão já são **LAZY** .

4.3. Estratégias de Cascata (CascadeType)

Ditam o que acontece com a entidade filha quando você executa uma ação na entidade pai:

- `CascadeType.PERSIST` : Ao salvar o pai, salva automaticamente os filhos que foram adicionados na lista.
- `CascadeType.MERGE` : Ao atualizar o pai, atualiza automaticamente os filhos.
- `CascadeType.REMOVE` : Ao deletar o pai, deleta automaticamente todos os seus filhos no banco.
- `CascadeType.ALL` : Aplica todas as operações acima simultaneamente.
- `orphanRemoval = true` : Se você remover um filho da lista Java (ex: `setor.getProcessos().remove(0)`), o JPA detecta que aquele filho virou "órfão" e gera um `DELETE FROM processos WHERE id = ?` no banco!

4.4. A Armadilha Mortal do Lombok `@Data` em Entidades

O Lombok é excelente, mas usar `@Data` em entidades com relacionamentos bidirecionais causa uma falha clássica:

A anotação `@Data` gera automaticamente o método `toString()` e `hashCode()` incluindo todos os atributos. Se `Setor` tem uma lista de `Processos` e `Processo` tem um `Setor`, quando o log tentar imprimir o objeto, o `Setor` chama o `toString` do `Processo`, que chama o `toString` do `Setor`... gerando um `StackOverflowError` imediato que derruba a JVM!

Boa Prática Corporativa Obrigatória:

Em entidades JPA, substitua o `@Data` por:

`@Getter`

`@Setter`

`@NoArgsConstructor`

E implemente `equals()` e `hashCode()` baseados unicamente no ID (ou use `@EqualsAndHashCode(of = "id")`).

Capítulo 5: Spring Data Repositories Ninja

Os Repositórios do Spring Data são onde a produtividade atinge o ápice. Você declara apenas interfaces e o Spring sintetiza as implementações em tempo de execução via Dynamic Proxies.

5.1. A Hierarquia de Interfaces do Spring Data

- `Repository<T, ID>` : Interface marcadora raiz (sem métodos).
- `CrudRepository<T, ID>` : Fornece os 11 métodos básicos de CRUD (save, findById, delete, count...).
- `PagingAndSortingRepository<T, ID>` : Adiciona suporte a paginação (`findAll(Pageable)`) e ordenação (`findAll(Sort)`).
- `JpaRepository<T, ID>` : A interface mais completa. Herda todas as anteriores e adiciona métodos específicos do JPA (como `flush()`, `saveAndFlush()` e operações em lote). **Esta é a que você deve estender na grande maioria dos casos!**

5.2. Derived Query Methods (Consultas Mágicas por Nome)

O Spring Data analisa as palavras em inglês no nome do método e constrói o SQL automaticamente:

Método Declarado na Interface	SQL Traduzido pelo Spring Data JPA
<code>findByAtivoTrue()</code>	<code>SELECT * FROM tabela WHERE ativo = true;</code>
<code>findByEmail(String email)</code>	<code>SELECT * FROM tabela WHERE email = ?;</code>
<code>findByNomeContainingIgnoreCase(String n)</code>	<code>SELECT * FROM tabela WHERE UPPER(nome) LIKE UPPER('%' ? '%');</code>
<code>existsByNumeroProtocolo(String p)</code>	<code>SELECT count(*) > 0 FROM processos WHERE numero_protocolo = ?;</code>
<code>countBySetorAtualId(Long id)</code>	<code>SELECT count(*) FROM processos WHERE setor_atual_id = ?;</code>
<code>findByDataAberturaBetween(d1, d2)</code>	<code>SELECT * FROM processos WHERE data_abertura BETWEEN ? AND ?;</code>
<code>findByStatusIn(List<Status> lista)</code>	<code>SELECT * FROM processos WHERE status IN (?, ?, ?);</code>
<code>findByOrgaoIdOrderByNomeAsc(Long id)</code>	<code>SELECT * FROM setores WHERE orgao_id = ? ORDER BY nome ASC;</code>
<code>deleteByAtivoFalse()</code>	<code>DELETE FROM tabela WHERE ativo = false;</code>

5.3. Paginação e Ordenação Profissional

Nunca retorne `List<Entidade>` em endpoints de listagem de sistemas reais. Se o banco tiver 50.000 registros, você esgota a memória RAM do servidor. Use `Pageable` :

```
// 1. No Repository:
public interface ProcessoRepository extends JpaRepository<Processo, Long> {
    Page<Processo> findByStatusProcesso(StatusProcesso status, Pageable pageable);
}

// 2. No Controller: O Spring injeta page, size e sort da URL automaticamente
// Exemplo de chamada: GET /api/v1/processos?page=0&size=10&sort=dataAbertura,desc
@GetMapping
public ResponseEntity<Page<ProcessoResponse>> listar(
    @RequestParam StatusProcesso status,
    @PageableDefault(size = 20, sort = "id", direction = Sort.Direction.DESC) Pageable pageable) {

    Page<ProcessoResponse> pagina = processoService.listarPorStatus(status, pageable);
    return ResponseEntity.ok(pagina);
}
```

Diferença entre Page<T> e Slice<T> :

- Page<T> : Executa uma query adicional de COUNT(*) para saber o número total de páginas (ótimo para tabelas com paginação tradicional 1, 2, 3...).
- Slice<T> : NÃO executa COUNT. Ele apenas sabe se "existe uma próxima página" buscando size + 1 registros (perfeito para telas de **Scroll Infinito** em mobile e redes sociais, economizando consultas de count pesadas).

5.4. Consultas Customizadas com @Query (JPQL vs SQL Nativo)

Quando a regra de busca for complexa demais para um nome de método derivado, usamos a anotação `@Query`.

```
public interface ProcessoRepository extends JpaRepository<Processo, Long> {

    // 1. JPQL (Java Persistence Query Language) - FOCADO EM OBJETOS E CLASSES!
    @Query("SELECT p FROM Processo p WHERE p.setorAtual.orgao.id = :orgaoId AND p.statusProcesso = :status OR")
    List<Processo> buscarPorOrgaoEStatus(
        @Param("orgaoId") Long orgaoId,
        @Param("status") StatusProcesso status
    );

    // 2. SQL NATIVO (nativeQuery = true) - FOCADO EM TABELAS FÍSICAS DO POSTGRESQL!
    // Use quando precisar de funções exclusivas do banco (ex: JSONB, ILIKE complexo, Window Functions)
    @Query(value = "SELECT * FROM processos p WHERE p.descricao ILIKE CONCAT('%', :termo, '%') LIMIT 50", nativeQuery = true)
    List<Processo> buscaTextualNativa(@Param("termo") String termo);
}
```

5.5. Operações de Modificação em Massa: @Modifying

Por padrão, o `@Query` espera um comando `SELECT`. Se você quiser rodar um `UPDATE` ou `DELETE` em massa diretamente no banco, você DEVE adicionar a anotação `@Modifying`:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    @Modifying(clearAutomatically = true) // Limpa o cache do EntityManager logo após o update
    @Transactional
    @Query("UPDATE Usuario u SET u.ativo = false WHERE u.dataAtualizacao < :dataLimite")
    int desativarUsuariosInativos(@Param("dataLimite") LocalDate dataLimite);
}
```

5.6. Projeções de Alta Performance (Interface e DTO)

Se você só precisa exibir o `id` e o `nome` de 10.000 usuários em uma caixa de seleção (Select Dropdown), você NÃO deve carregar a entidade completa com 30 colunas. Use uma **Projeção baseada em DTO**:

```
// 1. O Record DTO leve:
public record UsuarioResumoDTO(Long id, String nome, String email) {}

// 2. A Query usando o construtor do Record diretamente no JPQL:
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    @Query("SELECT new br.com.sigep.sigep.application.dto.usuario.UsuarioResumoDTO(u.id, u.nome, u.email) FROM Usuario u")
    List<UsuarioResumoDTO> buscarApenasResumos();
}
```

Capítulo 6: Gestão Transacional com @Transactional

A anotação `@Transactional` gerencia as fronteiras da transação no banco de dados. Sem ela, você corre o risco de deixar o banco inconsistente (ex: debitou o dinheiro da conta A, mas o sistema caiu antes de creditar na conta B).

6.1. Otimização Obrigatória: `@Transactional(readOnly = true)`

Em métodos que realizam apenas consultas (`SELECT`), use sempre `readOnly = true` :

```
@Transactional(readOnly = true)
public ProcessoResponse buscarPorId(Long id) { ... }
```

Ganhos de Performance Absurdos:

1. O Hibernate **desabilita o Dirty Checking**. Ele não precisa guardar snapshot em memória nem comparar atributos no final. Isso economiza até 40% de memória RAM e ciclos de CPU.
2. O driver JDBC pode encaminhar essa consulta automaticamente para réplicas de leitura (*Read Replicas*) do PostgreSQL em vez de sobrecarregar o banco primário de escrita.

6.2. Regras de Rollback

Por padrão, o Spring **só faz Rollback em exceções NÃO verificadas** (que estendem `RuntimeException` ou `Error`). Se o seu código lançar uma checked exception (ex: `IOException` , `SQLException`), o Spring fará o COMMIT mesmo com erro!

```
// A BOA PRÁTICA: Forçar rollback para QUALQUER exceção que ocorra!
@Transactional(rollbackFor = Exception.class)
public void tramitarProcessoComplexo(...) { ... }
```

Capítulo 7: Anatomia Prática do Projeto SIGEP

No seu projeto real `sigep-back` , o Spring Data JPA foi estruturado com as seguintes entidades conectadas:

MAPEAMENTO DE RELACIONAMENTOS DO SIGEP:

Orgao (1): Matriz governamental (ex: Secretaria de Fazenda)

▲ (1 Órgão tem Muitos Setores)

Setor (N): Divisão interna com `@ManyToOne Orgao`

▲ (1 Setor recebe Muitos Processos)

Processo (N): Entidade central com `@ManyToOne Usuario responsavel` e `@ManyToOne Setor setorAtual`

▼ (1 Processo gera Vários Históricos)

HistoricoTramitacao (N): Rastreabilidade imutável com `setorOrigem`, `setorDestino` e `data`

Capítulo 8: Blueprint Universal de Persistência

Para qualquer novo projeto que você criar na vida (ex: e-commerce, streaming, saúde, logística), siga este padrão imutável de 4 camadas:

1. Camada de Domínio (`domain/model`): Entidades com `@Entity`, `@Id`, `@Column` e `FetchType.LAZY`. Construtores expressivos e métodos de negócio ricos (ex: `processo.tramitar()`).

2. Camada de Repositório (`infrastructure/repository`): Interfaces estendendo `JpaRepository<Entidade, ID>` com consultas derivadas e JPQL enxuto.

3. Camada de Contrato (`application/dto`): Records Java imutáveis para requests e responses, protegendo suas entidades do mundo externo.

4. Camada de Aplicação (`application/service`): Métodos anotados com `@Transactional` coordenando persistência, dirty checking e regras de negócio.

Capítulo 9: As 4 Perguntas de Ouro do Arquiteto

RESUMO ARQUITETURAL: SPRING DATA JPA

- 1. Para que serve?** Mapear o mundo orientado a objetos do Java para o mundo relacional de tabelas SQL, automatizando CRUDs e queries.
- 2. Para que usar?** Para eliminar mais de 80% do código repetitivo de banco de dados (JDBC/SQL manual), ganhar paginação pronta e controle transacional robusto.
- 3. Quando usar?** Em quase 100% das aplicações empresariais que utilizam bancos de dados relacionais (PostgreSQL, MySQL, Oracle, SQL Server).
- 4. Devo usar em outros projetos?** Sim! É a tecnologia de persistência mais exigida, madura e consolidada de todo o ecossistema Java mundial.

Fim do Manual Definitivo de Spring Data JPA & Hibernate

Criado para Victor Zendem como guia de estudo, consulta contínua e arquitetura em Java.